FAQ-15: How Does DPL Work?

> **Series:** FAQ — Frequently Asked Questions | **Reference:** 15 — How Does DPL Work? | **Created:** July 2026 | **Last Updated:** 10/15/2026

Overview

Every log line that arrives as an undifferentiated blob of text eventually meets the same question: *how do I get the status code, the customer ID, the duration out of this string and into a field I can filter, group, and alert on?* In Dynatrace the answer is the **Dynatrace Pattern Language (DPL)** — a purpose-built pattern language that replaces the regular expressions most teams arrive with.

DPL is easy to start with and easy to be quietly wrong in. A pattern that looks correct returns `null` for every record. A pattern that returns values returns the *wrong* values, silently, with no error anywhere. A masking rule that appears to redact PII does nothing at all. All three are ordinary outcomes of misunderstanding two or three specific rules, and none of them announce themselves.

This entry is the mechanics deep-dive: **what a DPL pattern actually is, how it compares to the regular expressions you already know, the full matcher catalog, the matching rules that decide whether your pattern succeeds, the five DQL surfaces that consume DPL, where in the data path to parse, and how to diagnose a pattern that isn't working.** It complements **OPMIG-06** (*Processing, Parsing & Transformation*), which applies DPL inside an OpenPipeline pipeline, and **OPLOGS-05** (*Querying & Parsing Logs*), which applies it at query time. This entry is the language itself.

Nearly everyone arrives at DPL fluent in regex, so § 3 handles that head-on: exactly one construct is regex-compatible by design, several translate cleanly, and six pieces of regex intuition are actively wrong — most importantly that DPL does no backtracking, which turns a working regex into a pattern that matches nothing.

A note on how this was written. DPL's observable behavior is not fully described by its documentation, and in one place — how far the open-ended quantifiers `+`, `*` and `{n,}` reach (§ 6) — the two disagree. Every behavioral claim below was executed against a live Dynatrace tenant on **07/20/2026** (examples changed since were re-run on the date they

carry) and is labeled **live-verified**; documented-but-unverified claims are cited to the docs and labeled as such. Where the two conflict, both are shown. Examples use `data record(...)`, which fabricates records inline — so **every example here is runnable and scans zero bytes**, needing no log data and incurring no query consumption.

Table of Contents

1. Short Answer
2. The Pattern Model
3. Coming from Regex
4. The Matcher Catalog
5. Why Your Pattern Returns Null — or Worse, the Wrong Value
6. Quantifiers, Optionals, Alternation, Lookaround
7. The Five Surfaces That Consume DPL
8. DPL Beyond Logs
9. Where to Parse — Ingest vs. Query
10. Diagnosing a Failed Pattern
11. DPL Architect
12. Recommended Approach
13. Common Gotchas

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail. Every example runs as-is — `data record(...)` fabricates its own input, so no log data is required
Permissions	`storage:logs:read` to parse real logs; none at all for the `data record(...)` examples in this entry
OpenPipeline (optional)	`openpipeline.configurations.read` / `write` for the ingest-time half of § 9

Requirement	Details
Audience	Anyone writing a `parse` command, a Log Processing rule, or a masking processor; teams migrating Grok, regex, or SPL extractions
Related series	**OPMIG-06** (DPL inside a pipeline), **OPLOGS-05** (DPL at query time), **OPMIG-08** (masking), **OPIPE** (OpenPipeline beyond logs), **FAQ-09** (metric-vs-log query economics)

---

1. Short Answer

DPL is a sequence of matchers that must consume the input from left to right, in order, with nothing unmatched between them. You write the parts you want to keep as named matchers (`INT:status`), the parts you need to step over as literals (`' '`) or unnamed matchers, and the whole sequence has to line up against the text.

Five rules explain nearly every DPL problem:

1. **`parse` matches from the start of the string, contiguously.** It does not need to consume the whole line — trailing text is ignored — but it does not search forward for a starting point: `parse("xx 42", "INT:n")` is `null`, while `parse("42 xx", "INT:n")` is `"42"`. Any gap you don't account for breaks the match. `matchesPattern` is stricter still: the pattern must cover the *whole* string (§ 7). *(live-verified)*
2. **`LD` matches as *little* as possible; POSIX classes like `ALPHA` and `WORD` match as *much* as possible — and a bracket group like `[a-z]` matches exactly one character unless you quantify it.** These different defaults in one language cause most surprises. *(live-verified)*
3. **There is no backtracking.** A greedy matcher that over-consumes never gives characters back — the pattern fails instead. A quantified matcher must never be able to match the delimiter that follows it. *(live-verified)*
4. **Failure is silent.** A non-matching pattern yields `null`. An *ambiguous* pattern yields wrong values with no warning at all. There is no error either way.
5. **Use the dedicated matcher instead of hand-rolling it.** `HTTPDATE` is one token for what most teams write as a nine-part timestamp pattern, and `KVP` parses key-value text that teams usually attack one field at a time.

The single most important habit: **build patterns in DPL Architect against real sample records**, never in your head. See § 11.

> **Sources:** Dynatrace Pattern Language (DT docs), DPL Grammar (DT docs). **Derived:** the five-rule framing is an authoring synthesis — the docs describe the grammar, not this diagnostic ordering.

---

2. The Pattern Model

A DPL pattern is a whitespace-separated sequence of **matcher expressions**. Each one has the same documented shape:

```
[lookaround] MATCHER ['(' configuration ')'] [quantifier] [optional]
[':export_name']
```

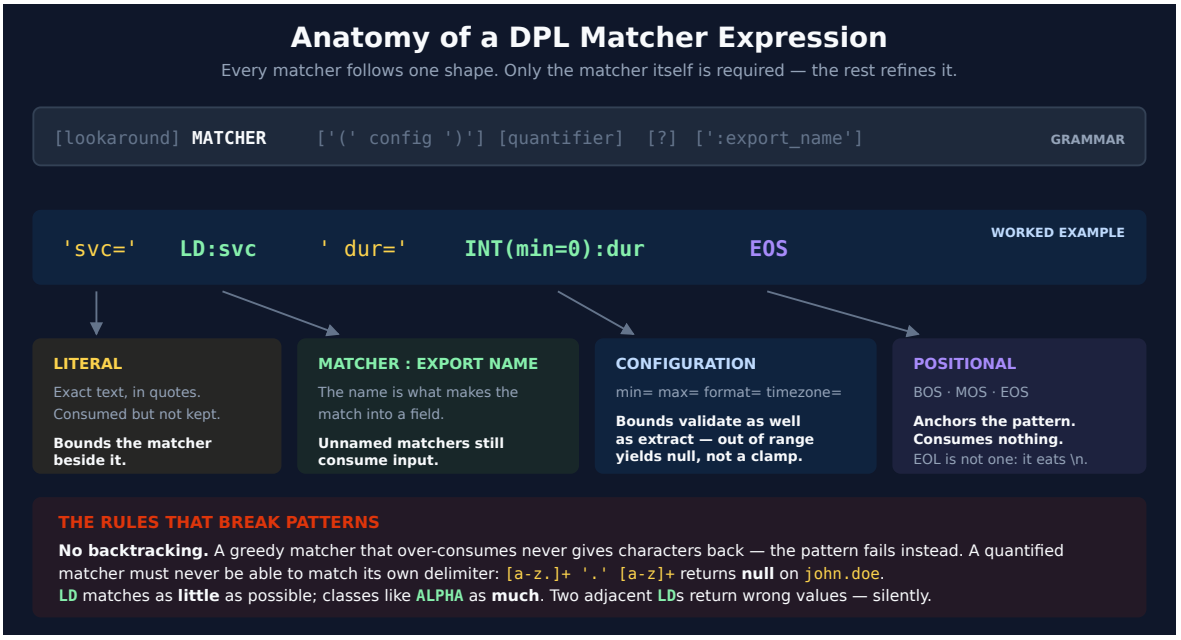
Only the matcher itself is required. Everything else refines it:

Part	Purpose	Example
`MATCHER`	What kind of text to consume	`INT`
`(config)`	Matcher-specific arguments	`TIMESTAMP(timezone='America/New_York')`
`{quantifier}`	How many repetitions	`ALPHA{2,8}`
`?`	Optional — may be absent	`(':' INT:port)?`
`:export_name`	Capture into a field of this name	`INT:status_code`
`>>` `<<`	Assert context without consuming it	`>>'ERROR'`

A matcher with no export name still consumes input — it just doesn't keep it. This is the mechanism for stepping over the parts of a line you don't care about, and it's the part that trips up people coming from regex, where an uncaptured group is optional noise. In DPL every matcher in the sequence must match; naming only decides whether the result is retained.

Literals are text in single or double quotes and match exactly: `'status='`. Escape an embedded quote with a backslash, or switch quote styles.

The anatomy of a real pattern



Macros keep long patterns readable

Repeated fragments can be named once and reused, which matters for syslog-style formats where every line shares a header:

```
$syslog_hdr = TIMESTAMP('MMM d HH:mm:ss'):ts ' ' LD:host;
$syslog_hdr ' ' LD:process ': ' LD:message EOS;
```

> ⚠️ **End a pattern with `EOS`, not `EOL`.** `EOL` is not an anchor — it matches and consumes a line-feed character, so a pattern ending in `EOL` returns `null` on any record without a trailing `\n`, which is almost every single-line log record. Both examples above return `null` with `EOL` and parse with `EOS`. *(live-verified 10/02/2026)*

```
data record(t = "svc=checkout dur=42")
| fieldsAdd with_eol = parse(t, "'svc=' LD:svc ' dur=' INT(min=0):dur EOL"),
            with_eos = parse(t, "'svc=' LD:svc ' dur=' INT(min=0):dur EOS")
// live-verified 10/02/2026: with_eol = null, with_eos = {svc: "checkout",
dur: 42}
```

> **Sources:**
>
> - DPL Grammar (DT docs)
> - DPL Modifiers (DT docs)
> - Literal expressions (DT docs)
> - Macros (DT docs)
> - Lines and strings (DT docs) — *"EOL, LF Matches the single line feed character"*
> - Positional matchers (DT docs) — *"Positional matchers always refer to the absolute start or end of the input string, regardless of newlines."*

---

3. Coming from Regex

Most people meet DPL already fluent in regular expressions, and the resemblance is close enough to be dangerous. DPL is **not** a regex dialect. Exactly one part of it is regex-compatible by design, a handful of constructs translate cleanly, and a few pieces of regex intuition are actively wrong.

3.1 The one part that transfers verbatim: character groups

Character groups are the deliberate bridge. The documentation states it plainly — the syntax *"is compatible with Regular Expression Character Class"* — and DPL goes slightly further than regex by accepting **both** `^` and `!` as the negation character:

```
data record(t = "abc123")
| fieldsAdd caret = parse(t, "[^0-9]+:x"), bang = parse(t, "[!0-9]+:x")
// live-verified: both return "abc"
```

Ranges (`[0-9]`), sets (`[abc]`), and POSIX bracket forms (`[:lower:]`) all work as a regex user expects. **If you know regex character classes, you already know this part of DPL.**

3.2 Translation table

Regex	DPL	Notes
`[a-z]` `[0-9]` `[abc]`	identical	Documented as regex-compatible
`[^0-9]`	`[^0-9]` or `[!0-9]`	Both negation characters accepted
`\d` `\w` `\s` `\S`	`DIGIT` `WORD` `SPACE` `NSPACE`	**Backslash shorthand is a syntax error**
`.` (any character)	*no direct equivalent*	See § 3.3 — `'.'` is a literal dot
`.*` / `.+` (rest of line)	`LD`	**Semantics differ — see § 3.3**
`[\s\S]*` (across lines)	`DATA`	
`x*` `x+` `x?`	identical syntax	Open-ended forms are bounded — see § 6
`x{2,5}` `x{3}`	**only on some matchers**	Numeric and semantic matchers reject `{n}` — see § 3.3
Backtracking	**none**	The biggest difference in this table — see § 3.3
`(a|b)`	`(a|b)`	Lazy, left-to-right; losers export `null`
`(?=…)` `(?!…)`	`>>` `!>>`	
`(?<=…)` `(?<!…)`	`<<` `!<<`	64-byte look-behind window
`^` `$` anchors	`BOS` `EOS`	`EOL` is not an anchor — it consumes a `\n` (§ 2)
`(?<name>…)` named capture	`MATCHER:name`	Naming *is* how you capture
`\.` escaped literal	`'.'` quoted literal	All literals are quoted, never escaped
Backreference `\1`	—	Not available
Inline flags `(?i)`	—	No general case-insensitivity modifier

3.3 Six places regex intuition is wrong

All six are live-verified, and each produces a different kind of failure. The first one is the most important sentence in this entry.

1. There is no backtracking. A regex engine that over-consumes will give characters back and try again. **DPL will not.** A quantified matcher takes everything it can, and if that leaves the next matcher with nothing to match, the whole pattern fails — silently:

```
data record(a = "john.doe@corp.com")
| fieldsAdd broken    = parse(a, "[a-z.]+:x '.' [a-z]+:y"),
            tightened = parse(a, "[a-z]+:x '.' [a-z]+:y")
// live-verified: broken = null,  tightened = {x: "john", y: "doe"}
```

`[a-z.]+` includes the dot, so it swallows `john.doe` entirely and the `'.'` literal has nothing left. Every regex engine in common use would backtrack one character and match. DPL returns `null`.

> **The rule that follows: a quantified matcher must never be able to match its own delimiter.** Exclude the delimiter from the character class. This single rule prevents more DPL failures than any other.

This is also why porting a working regex can produce a pattern that never matches anything — the regex was relying on backtracking you didn't know it was using.

2. `.` is not a wildcard — it's a literal dot. In DPL a quoted `'.'` matches one actual period and nothing else:

```
data record(x = "x")
| fieldsAdd hit  = parse("a.b", "LD:l '.' LD:r"),
            miss = parse("axb", "LD:l '.' LD:r")
// live-verified: hit = {l:"a", r:"b"},  miss = null
```

There is no bare `.` metacharacter. To match "any character" use a character group or a POSIX class.

3. `.*` is greedy; its DPL counterpart `LD` is shortest-match. Regex `.*` grabs everything and backs off. `LD` stops at the *first* occurrence of whatever follows it, not the last:

```
data record(x = "x")
| fieldsAdd p = parse("a:b:c", "LD:x ':' LD:y")
// live-verified: {x: "a", y: "b:c"} — first colon wins, not the last
```

A regex `(.*):(.*)` on the same input splits at the **last** colon. Ported literally, the two disagree on every line with more than one delimiter. Covered further in § 5.

4. Backslash shorthand doesn't exist. `\d+` isn't an unsupported feature, it's a parse failure:

```
ERROR: The parsing pattern is invalid. Syntax error: mismatched input '\'
```

Use `DIGIT+` or `[0-9]+`.

5. Quantifier support is per-matcher, not universal. Regex lets you write `\d{3}`. DPL does not let you write `INT{3}` — numeric and semantic matchers accept only `*` and `+`:

```
INT{3}   → ERROR: INT only supports '*' (nullable) and '+' (not nullable)
quantifiers
```

To match exactly three digits, use a character class: `[0-9]{3}:code`. Note the type consequence — `[0-9]{3}` yields a **string**, while `INT` yields a number, so add `toLong()` if the numeric type matters. Sequence and alternation groups reject quantifiers entirely.

This one at least fails loudly. It is still worth knowing before you write a fixed-width pattern for an SSN, a zip code, or a card fragment.

6. Nothing is unbounded. Regex `+` means "one or more, however many." DPL `+` is capped — the docs say at 4,096 repetitions, and on a live tenant it reached 8,192 (§ 6). A regex that works on a 10 KB line has a DPL translation that silently stops short. For long fields set an explicit maximum, e.g. `DATA{1,20000}`.

3.4 Coming from Grok

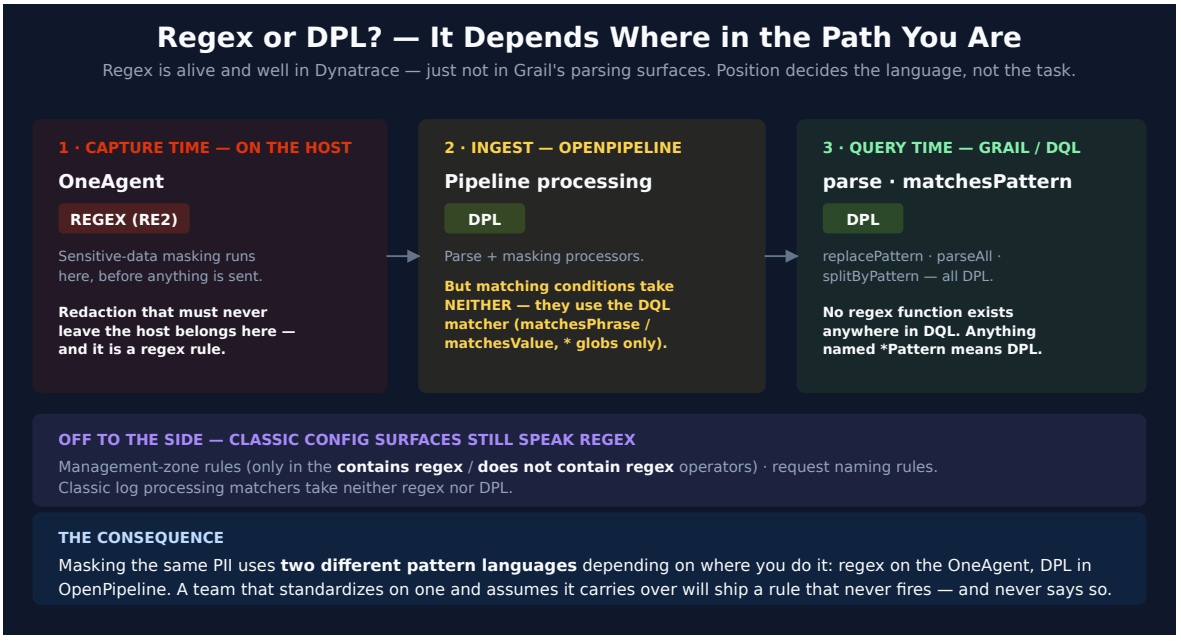
Grok is regex underneath, and its `%{SYNTAX:name}` form maps almost one-to-one onto DPL's `MATCHER:name` — `%{IP:client}` → `IPADDR:client`, `%{INT:n}` → `INT:n`, `%{GREEDYDATA:msg}` → `DATA:msg`. **OPMIG-06** carries the fuller Grok and Logstash-filter conversion tables; this entry doesn't duplicate them.

One caution when porting: `%{GREEDYDATA}` is greedy by name and by behavior, and `DATA` is not. Check the § 5 rules before trusting a mechanical substitution.

In community practice this is where most porting effort actually goes — verify converted patterns against real records rather than assuming table-driven equivalence. Dynatrace publishes no Grok migration guidance of its own, so the mapping above is community knowledge, not a documented contract.

3.5 Where regex is still the right tool in Dynatrace

This is the part that surprises people: **regex is alive and well in Dynatrace — just not in Grail's parsing surfaces.** Which language a surface speaks depends on *where in the pipeline it sits*, not on what it does.



The practical consequence is worth stating directly: **masking the same PII uses two different pattern languages depending on where you mask it.** Redacting at capture time on the OneAgent is a regex rule; redacting the same field in OpenPipeline is DPL. A team that standardizes on one and assumes it applies to the other will ship a rule that never fires. **OPMIG-08** covers the OpenPipeline side.

3.6 Functions that sound like regex but aren't

The naming actively misleads here. Everything with "Pattern" in it takes **DPL**:

Function	Actually takes
`matchesPattern`, `replacePattern`, `splitByPattern`	**DPL** — despite the name

Function	Actually takes
`like`	SQL wildcards — `%` (any sequence), `_` (one character)
`matchesValue`	`*` wildcard; **no mid-value wildcards**
`matchesPhrase`	`*` wildcard scoped to a token; **no mid-string wildcards**
`contains`, `startsWith`, `endsWith`	No wildcards at all — plain substring

```
data record(u = "GET /api/users")
| fieldsAdd mv = matchesValue(u, "GET*"), mp = matchesPhrase(u, "api*")
// live-verified: both true
```

3.7 A note on the regex you do write

Where Dynatrace *does* accept regex, its own documentation is unusually cautionary: quantified or repeated groups are not allowed, backreferences are not supported, greedy captures should be replaced with lazy or possessive forms, and the docs give a worked example of `(.*)+b` consuming more than ten seconds against a string with no `b` — the classic catastrophic-backtracking blowup.

DPL cannot express that failure mode: every quantifier is bounded and look-behind is capped at 64 bytes. Whether that boundedness was *chosen* to avoid backtracking is not something Dynatrace states anywhere, so treat the connection as an observation about the design rather than a documented rationale.

In community practice the Grok mappings in § 3.4 are the accepted correspondences; no Dynatrace documentation covers Grok, so verify each against DPL Architect before relying on it.

> **Sources:**
>
> - Lines and strings (DT docs) — "The syntax is compatible with Regular Expression Character Class"; negation via `^` or `!`
> - Dynatrace Pattern Language (DT docs) — character groups described as "Regular Expression compatible"
> - Regular expressions in Dynatrace (DT docs) — the catastrophic-backtracking warning, the `(.*)+b` example, and the no-backreferences / no-repeated-groups restrictions

- Sensitive data masking (DT docs) — *"Search expression: A regular expression to match the string that you want to mask. Use the regular expression format."* The linked words "regular expression" point to RE2 syntax (Google GitHub)
- Management zone rules (DT docs) — regex permitted only in the `contains regex` / `does not contain regex` operators
- DQL matcher in OpenPipeline (DT docs) — matching conditions accept neither regex nor DPL
- String functions (DT docs) — `like` SQL wildcards; `matchesValue` / `matchesPhrase` wildcard scope
- **Derived:** § 3.5's two-languages contrast and § 3.7's bounded-quantifier observation are both inferences — no page draws either, and Dynatrace publishes no DPL design rationale

---

4. The Matcher Catalog

The catalog below is the DPL Grammar reference, reorganized by what you reach for it to do. **Matcher names are case-sensitive and exact** — a name that isn't in this catalog is a syntax error, not a silent no-match (see § 10).

Numeric and boolean

Matcher	Matches	Config
`INT`, `INTEGER`	Integers, −2147483648 … 2147483647	`min=`, `max=`
`LONG`	Integers, long range	`min=`, `max=`
`HEXINT` / `HEXLONG`	Hex notation	—
`FLOAT` / `DOUBLE`	Decimal or scientific notation	`min=`, `max=`
`CFLOAT` / `CDOUBLE`	As above, comma as decimal separator	`min=`, `max=`
`BOOLEAN`, `BOOL`	`true` / `false`, case-insensitive	—

⚠️ **Numeric matchers accept only the `*` and `+` quantifiers** — `INT{3}` is rejected outright. Use a character class for fixed-width digits. See § 6. *(live-verified)*

`min=` / `max=` **make a matcher a validator, not just an extractor.** An out-of-range value doesn't clamp — the match fails and the field is `null`:

```
data record(t = "404")
| fieldsAdd in_range = parse(t, "INT(min=200, max=299):code"),
            no_bound = parse(t, "INT:code")
// live-verified: in_range = null, no_bound = 404
```

That makes bounded matchers a neat way to extract *and* filter in one step — `INT(min=500):code` captures only server errors.

Strings and line data

Matcher	Matches	Default quantifier
`LD`, `LDATA`	Any characters up to the next matcher, **within one line**	`{1,4096}`
`DATA`	Any characters up to the next matcher, **across lines**	`{1,4096}`
`SQS` / `DQS`	Single- / double-quoted strings, backslash escaping	`{1,4096}`
`CSVSQS` / `CSVDQS`	Quoted strings with CSV doubling (`""`)	`{1,4096}`
`STRING`	Quoted strings or character groups	*not documented*

`DQS` deserves attention: the Apache and Nginx patterns most teams write hand-roll quoted fields as `'"' LD:x '"'`, which breaks on any escaped quote inside the value. `DQS:x` handles that correctly in one token.

Character classes

POSIX classes, usable directly as matchers: `ALNUM`, `ALPHA`, `BLANK`, `CNTRL`, `DIGIT`, `GRAPH`, `LOWER`, `PRINT`, `PUNCT`, `SPACE`, `NSPACE` (non-space), `UPPER`, `XDIGIT`, `ASCII`, `WORD`. Custom groups use bracket syntax — `[0-9]`, `[a-fA-F]`, negated with `^` or `!`.

> ⚠️ **POSIX classes and bracket groups have different defaults.** A POSIX class matches 1 to 4,096 characters by default; a bracket group matches **exactly one**

> character unless you add a quantifier. On input `abcdef:x`, `ALPHA:x` returns **`abcdef`**, `[a-z]:x` returns **`a`**, and `[a-z]+:x` returns `abcdef`; `ALPHA{2}:x` returns `ab`. *(live-verified 07/20/2026 and 09/28/2026)* A bracket group written as if it were a POSIX class silently captures one character — **state the quantifier explicitly** whenever the length matters.

```
data record(t = "abcdef:x")
| fieldsAdd posix = parse(t, "ALPHA:x"), bracket = parse(t, "[a-z]:x"),
bracket_plus = parse(t, "[a-z]+:x")
// live-verified 09/28/2026: posix = "abcdef", bracket = "a", bracket_plus =
"abcdef"
```

Date and time

Matcher	Format
`TIMESTAMP`, `TIME`	Flexible; default `yyyy-MM-dd HH:mm:ss`. Config: `format=`, `timezone=`, `locale=`
`ISO8601`	`yyyy-MM-ddTHH:mm:ssZ`
`JSONTIMESTAMP`	`yyyy-MM-ddTHH:mm:ss.SSSZ`
`HTTPDATE`	`dd/MMM/yyyy:HH:mm:ss Z`

All output is normalized to UTC. Two verified behaviors worth knowing:

```
data record(t = "12/Dec/2024:10:30:45 +0000")
| fieldsAdd short = parse(t, "HTTPDATE:ts"),
            long  = parse(t, "TIMESTAMP('dd/MMM/yyyy:HH:mm:ss Z'):ts")
// live-verified: both return 2024-12-12T10:30:45.000000000Z — identical
```

`HTTPDATE` **is** the Apache/Nginx timestamp format. Any pattern spelling it out by hand is doing work a single token already does.

```
data record(t = "2024-12-12 10:30:45")
| fieldsAdd utc = parse(t, "TIMESTAMP(timezone='America/New_York'):ts")
// live-verified: 2024-12-12T15:30:45Z — the timezone config converts, it does
not merely label
```

Network

`IP` / `IPADDR` (v4 or v6 — `IP` is the name the docs now lead with, `IPADDR` its alias), `IPV4` / `IPV4ADDR`, `IPV6` / `IPV6ADDR`. Quantifiers `*` and `+` only (no `{n}`), no configuration. **There is no CIDR or prefix matcher.**

These matchers are not validators. They read the longest valid leading portion of the address and stop, so an invalid address is **truncated to a shorter valid one rather than returning `null`**:

```
data record(t = "192.168.0.256")
| fieldsAdd ip = parse(t, "IPADDR:ip")
// live-verified 09/28/2026: ip = "192.168.0.25" — the out-of-range octet was cut, not rejected
```

If a pipeline must reject bad addresses, check the parsed value against the source (for example, that the parsed address is the whole token) rather than trusting a non-null result.

Structured data

Matcher	Syntax	Produces
`JSON`, `JSON_OBJECT`	`JSON:payload` or `JSON{ MATCHER:member, … }`	object
`JSON_ARRAY` / `JSON_VALUE`	`JSON_ARRAY{jsonValueType}`	array / value
`XML`, `XML_PLAIN`, `XML_VERBOSE`	`XML:doc`	object
`KVP`	`KVP{ …:key … :value }:name`	object
`ARRAY`	`ARRAY{ … }{n,m}:name`	array
`STRUCTURE`	`STRUCTURE{ … }:name`	record
`ENUM`	`ENUM{'A'=1,'B'=2}:name`	integer

These four are the most under-used matchers in the language, and all are live-verified:

```
data record(t = "a=1, b=2, c=3")
| parse t, "KVP{ALNUM+:key '=' INT:value ', '?}:kv"
// live-verified 10/02/2026: kv = {a: 1, b: 2, c: 3} — values typed long,
because INT:value
```

```
data record(x = "x")
| fieldsAdd sev   = parse("WARN",
"ENUM{'INFO'=1,'WARN'=2,'ERROR'=3}:sev"),
           nums  = parse("1,2,3",    "ARRAY{INT:v ','?}{1,10}:nums"),
           point = parse("10.5:200", "STRUCTURE{DOUBLE:lat ':' INT:port}:s")
// live-verified: sev = 2, nums = [1,2,3], point = {lat: 10.5, port: 200}
```

> ⚠️ **`ARRAY` requires an explicit quantifier.** Omitting it is a hard syntax error, not a default: `The parsing pattern is invalid. ARRAY requires a quantifier expression`. *(live-verified)*

`JSON` is worth a special note: if a log line is entirely JSON, `parse content, "JSON:j"` gives you the whole document as a navigable object in one step, and `j[fieldName]` reaches into it — usually better than extracting fields one at a time.

Positional and specialized

- **`BOS` / `BOF`**, **`MOS` / `MOF`**, **`EOS` / `EOF`** — beginning, middle, end of string. Anchors; they consume nothing.
- **`EOL` / `LF`**, **`CR`**, **`EOLWIN` / `WINEOL`** — line terminators.
- **`CREDITCARD`** — validates the Luhn checksum across formats and encodings; a failed checksum yields `null`. This is the matcher behind PCI masking rules.
- **`SMARTSCAPEID`** — Smartscape entity ID form.

> ⚠️ **There is no `EMAIL` matcher.** It is a natural guess and it is not real — `parse t, "EMAIL:mail"` fails with `Named pattern element 'EMAIL' is not valid`. *(live-verified)* Email extraction and masking must be hand-rolled from character classes and literals. Other plausible-sounding names that **do not exist**: `URI`, `URL`, `HEXNUM` (use `HEXINT` / `HEXLONG`), `IPPREFIX`, `HTTPMETHOD`, `DATETIME` (use `TIMESTAMP` / `TIME`), `LINE`, `CHAR`, `BYTE`, `BOM`.
>
> **Sources:**
>
> - DPL Grammar (DT docs)

- Numeric matchers (DT docs)
- Lines and strings (DT docs) — *"By default, POSIX character classes match between 1 and 4096 characters."*; *"By default, [...] matches exactly one character from the group"*
- Time and date (DT docs)
- Network matchers (DT docs) — *"Some invalid or non-standard inputs are therefore truncated to a shorter valid address instead of returning null"* (page updated Aug 06, 2026)
- Key-value pairs (DT docs)
- Positional matchers (DT docs)
- XML matchers (DT docs) — absent from the grammar table; reachable only by direct link
- **Tenant observation (07/20/2026; spot-checked 09/28/2026):** each name on the "does not exist" list was rejected with `Named pattern element '<name>' is not valid` when parsed against a live tenant (09/28 re-check: `URI`, `HEXNUM`, `DATETIME` rejected; `LD` as control parsed); no documentation page states the list

---

5. Why Your Pattern Returns Null — or Worse, the Wrong Value

This section is the reason the entry exists. DPL has **two** failure modes, and the dangerous one produces data rather than nothing.

Rule 1 — `LD` is minimal; POSIX classes are greedy

`LD` consumes **as little as possible** until the next matcher can succeed. POSIX classes such as `WORD` consume **as much as possible** (a bracket group, by contrast, takes one character unless quantified — § 4). Both live-verified, and the asymmetry is the root of most confusion:

```
data record(t = "key_abcdef rest")
| fieldsAdd via_class = parse(t, "'key_' WORD:w"),
            via_ld    = parse(t, "'key_' LD:w ' '")
// live-verified: via_class = "abcdef"  (greedy — took the whole word)
//                via_ld    = "abcdef"  (minimal — stopped at the space
delimiter)
```

Both give the right answer here, for opposite reasons. Change the input and they diverge — which is exactly why the next rule matters.

Rule 2 — greedy matchers never give characters back

DPL has **no backtracking**. When a quantified matcher over-consumes, it does not retry with less. The pattern just fails:

```
data record(a = "john.doe@corp.com")
| fieldsAdd broken    = parse(a, "[a-z.]+:x '.' [a-z]+:y"),
            tightened = parse(a, "[a-z]+:x '.' [a-z]+:y")
// live-verified: broken = null,  tightened = {x: "john", y: "doe"}
```

`[a-z.]+` can match the dot, so it eats `john.doe` and leaves the `'.'` literal with nothing. A regex engine backtracks one character and succeeds; DPL returns `null`.

A quantified matcher must never be able to match the delimiter that follows it. Exclude the delimiter from the class. This is the highest-yield rule in this section, and it is the reason a regex ported verbatim can match nothing at all — the original was quietly relying on backtracking.

Rule 3 — `LD` without a following delimiter takes one character

Because `LD` is minimal, **its minimum is what it takes when nothing forces it wider**. Two adjacent `LD` matchers are the classic trap:

```
data record(t = "alpha beta gamma")
| fieldsAdd broken   = parse(t, "LD:x LD:y"),
            anchored = parse(t, "LD:x ' ' LD:y ' '")
// live-verified: broken   = {x: "a",     y: "lpha beta gamma"}   ← no error,
just wrong
//                anchored = {x: "alpha", y: "beta"}              ← correct
```

`broken` did not fail. It produced two populated fields, one of which is a single stray character, and nothing in the query, the pipeline, or the UI flags it. A pipeline built on that pattern will happily write `x = "a"` for millions of records. **Always put a literal or a non-optional matcher between two `LD`s.**

Rule 4 — `LD` stops at the line; `DATA` crosses lines

```
data record(t = "line one\nline two\nline three")
| fieldsAdd ld   = parse(t, "'line ' LD:x"),
            data = parse(t, "'line ' DATA:x EOS")
// live-verified: ld   = "one"
//                data = "one\nline two\nline three"
```

This is the whole reason `DATA` exists, and it is how you capture a Java stack trace: the message is `LD`, the trace body is `DATA`.

Rule 5 — an unmatched literal yields `null`, full stop

```
data record(t = "alpha beta gamma")
| fieldsAdd nope = parse(t, "'zzz' LD:x")
// live-verified: nope = null
```

No error, no warning. When every record comes back `null`, suspect a literal that doesn't appear in the text — a different separator, a tab instead of a space, a field order that varies.

Rule 6 — both `LD` and `DATA` cap at 4096 characters by default

The documented default quantifier is `{1,4096}`. On long lines — stack traces, embedded payloads, verbose access logs — a pattern that works on your samples can fail on production records simply because the text ran past the cap. Widen it explicitly: `DATA{1,20000}:body`.

> **Sources:** Lines and strings (DT docs) — documents the `{1,4096}` defaults, the line-vs-pattern scope distinction, and the rule that `LD` / `DATA` "must always be followed by a non-optional matcher expression". **Tenant observation (07/20/2026):** the minimal-vs-greedy behavior and the silent-wrong-value failure mode were reproduced by executing the patterns in this section against a live tenant; the documentation states the constraint but does not describe what happens when it is violated.

---

6. Quantifiers, Optionals, Alternation, Lookaround

Quantifiers

Syntax	Meaning
`{n}`	Exactly n
`{n,m}`	Between n and m
`{n,}`	At least n — capped (see below)
`{,m}`	Up to m
`*`	0 to the cap
`+`	1 to the cap

Every open-ended quantifier is capped — and the documented cap is not the observed one. The docs define `+` as *"at least 1 times and max 4096 times"*, and the same 4,096 for `*` and `{n,}`. On a live tenant (10/02/2026, zero-byte `data record` tests) the explicit open-ended forms reached **8,192**: on a 4,400-character run, `LD+`, `LD{1,}`, `[a]+` and `ALPHA+` all captured 4,400 characters, while the **unquantified** `LD` and `ALPHA` stopped at 4,096; on a 9,600-character run `LD+` and `ALPHA+` stopped at 8,192. Plan for the documented 4,096 and do not rely on the higher observed value — either way the cap is real.

What hitting the cap looks like depends on what follows the matcher. If the pattern must still match something after the capped matcher — a literal or `EOS` — the parse returns `null`. If the capped matcher is the last thing in the pattern, the common `… LD:message` shape, it returns the first 4,096 (or 8,192) characters and **no error**: a silently truncated field. Live-verified 10/15/2026: on 4,400 characters `LD:x` returned 4,096 characters while `LD:x EOS` and `LD:x '|'` returned `null`; on a longer run `LD+:x` returned 8,192 characters and `LD+:x EOS` returned `null`. For long fields set an explicit maximum (`DATA{1,20000}:body` captured all 9,600 characters); the docs allow up to 16,000,000 for `LD` and `DATA`.

> ⚠️ **Which quantifiers you may use depends on the matcher.** Counted forms (`{n}` , `{n,m}`) are not universal:

Matcher family	`*` `+`	`{n}` `{n,m}`
Character classes and character/token matchers (`[0-9]` , `WORD` , `ALPHA` , `LD` , `DATA` , `STRING`)	yes	yes
Numeric and semantic matchers (`INT` , `LONG` , `DOUBLE` , `FLOAT` , `IPADDR` , `TIMESTAMP` , `BOOLEAN` , `CREDITCARD`)	yes	**no**
Sequence groups `( … )` and alternation groups	**no**	**no**

`INT{3}` fails with `INT only supports '*' (nullable) and '+' (not nullable) quantifiers` . For a fixed-width number use a character class — `[0-9]{3}:code` — and remember it yields a **string**, not a number; wrap in `toLong()` if the type matters. *(live-verified)*

Optional matchers

`?` marks a matcher expression as optional and goes *after* any quantifier but *before* the `:name` export. Group it with its delimiter so the whole unit disappears together:

```
// Right — the colon vanishes with the port.
// "10.0.0.1 rest"      → {ip: 10.0.0.1}
// "10.0.0.1:8080 rest" → {ip: 10.0.0.1, port: 8080}
parse content, "IPADDR:ip (':' INT:port)?"

// Wrong — rejected before it ever runs. `?` cannot follow the export name:
// ERROR_IN_PARSING_PATTERN "Syntax error: extraneous input '?' expecting
&lt;EOF&gt;".
parse content, "IPADDR:ip ':' INT:port?"

// Worse — this one compiles, and then fails silently. The optional marker is
// placed legally, but the literal ':' is still mandatory outside the optional
// unit, so an address with no port extracts NOTHING and raises no error.
// "10.0.0.1 rest"      → {}            ← the silent case
// "10.0.0.1:8080 rest" → {ip: 10.0.0.1, port: 8080}
parse content, "IPADDR:ip ':' INT?:port"
```

The placement rule is the point: `?` binds to the **matcher**, not to the exported field. Put it after the export and the pattern is a compile error; put it correctly but leave the delimiter outside the optional unit and you get this notebook's own subject — a pattern that parses, runs, and quietly returns nothing. *(all three live-verified 08/27/2026)*

This is how you write one pattern for a log format with genuinely optional fields, instead of two patterns and a `coalesce`.

Alternation

`( a | b )` matches the first alternative that succeeds — a documented **lazy, left-to-right** strategy. Fields exported by the alternatives that didn't win are `null`, so order matters: put the most specific alternative first.

```
parse content, "('userId=' | 'user=' | 'uid=') LD:user"
```

Lookaround — assert without consuming

Four modifiers test the surrounding text without advancing the match position:

Modifier	Meaning
`>>`	Positive lookahead
`!>>`	Negative lookahead
`<<`	Positive lookbehind (**64-byte window**)
`!<<`	Negative lookbehind (64-byte window)

```
data record(t = "ERROR svc=api code=500 code=502")
| fieldsAdd ahead  = parse(t, "&gt;&gt;'ERROR' LD:lvl ' '"),
            behind = parse(t, "LD &lt;&lt;'svc=' NSPACE+:svcname ' '")
// live-verified: ahead = "ERROR", behind = "api"
```

Negative lookahead is the tidy way to say "parse this line only if it *isn't* a heartbeat" without a separate filter.

> **Sources:** DPL Modifiers (DT docs) — quantifier forms, *"+ specifies repetition with at least 1 times and max 4096 times"*, optional placement, and the 64-byte look-behind window; Lines and strings (DT docs) — LD/DATA *"Default: 4096 . Maximum: 16000000"*; Alternatives group (DT docs) — the lazy left-to-right strategy and `null` for losing alternatives.

---

7. The Five Surfaces That Consume DPL

Most teams know `parse`. There are five, and picking the right one removes a lot of awkward DQL.



`parse` — the command

```
parse expression, pattern [, preserveFieldsOnFailure] [, parsingPrerequisite]
```

Adds extracted fields directly to the record. Two optional parameters are almost unknown and both are useful:

- **`preserveFieldsOnFailure`** — keep existing field values when parsing fails instead of overwriting them with `null`.
- **`parsingPrerequisite`** — a condition deciding whether a record is parsed at all, so mixed-format streams don't have to be split first.

> ⚠️ **`parse` silently overrides existing fields of the same name.** DQL emits a warning — `The field `<fieldName>` overrides an existing field` — but the query still runs. Prefix your export names (`x_status`, not `status`) when parsing into a record that already has semantic fields.

`parse` — the function

The same DPL, used inline in `fieldsAdd`. Returns the value directly, or a record when the pattern exports several names. Every example in this entry uses this form.

`parseAll` — every occurrence

`parse` stops at the first match. When the thing you want repeats, `parseAll` returns all of them:

```
data record(t = "ERROR svc=api code=500 code=502")
| fieldsAdd codes = parseAll(t, "'code=' INT:c")
// live-verified 10/02/2026: codes = [500, 502] — an array of long
```

`matchesPattern` — DPL as a filter

`matchesPattern(expression, pattern)` returns a boolean. This is DPL used for classification rather than extraction, and it's far more precise than `contains()`:

```
fetch logs, from: -1h
| filter matchesPattern(content, "IPADDR ' - - [' DATA")
```

`matchesPattern` must match the whole string, not a prefix. Without the trailing `DATA`, the same filter returns `false` on every access-log line — live-verified 09/24/2026: 0 records against 11,175 with `DATA` over the same hour, and `matchesPattern("10.1.2.3 GET /x", "IPADDR")` is `false`. `DATA` absorbs the rest of the record, line breaks included; `LD` stops at the first line break, so it fails on multi-line content. `contains(content, "192.168")` matches a version string; this pattern matches only records that start with an actual address. In OpenPipeline, use `matchesPattern` inside a DQL processor — pipeline **matching conditions** accept only `matchesPhrase` / `matchesValue` (§ 3.5).

`replacePattern` — masking

`replacePattern(expression, pattern, replacement)` is the masking primitive:

```
fieldsAdd content = replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] CREDITCARD
!&gt;&gt;[0-9A-Za-z-]", "[PAN_REDACTED]")
```

Because masking depends on the pattern matching, **an invalid or non-matching pattern means no redaction happens** — and nothing tells you. Test every masking pattern

against sample data before deploying it. See § 13.

Bound the matcher. A bare `CREDITCARD` matches a Luhn-valid digit run wherever it sits, including inside an identifier: on `req 9cbcb510-8833-4981-87ab-1234567890ab ok` it returns `req 9cbcb[PAN_REDACTED]ab-1234567890ab ok`. On a validation tenant, 1,366 of the 1,541 log records a bare `CREDITCARD` changed in 15 minutes were IDs of that kind, not card numbers (10/12/2026). As an ingest processor it would have corrupted those IDs, forward-only. The negative lookbehind `!<<` and negative lookahead `!>>` above refuse a match that touches a letter, digit or hyphen, so the UUID is left intact while `4111111111111111`, `4111-1111-1111-1111`, `4111 1111 1111 1111` and `x=4111111111111111,y` are all still masked (live-verified 10/12/2026).

Partial masking uses a lookahead, not a back-reference. `replacePattern` has no back-references, but `>>` checks text without consuming it, so the checked text survives the replacement: `replacePattern("ssn 123-45-6789 rest", "!<<[0-9A-Za-z-] [0-9]{3} '-' [0-9]{2} '-' >>([0-9]{4} !>>[0-9A-Za-z-])", "***-**-")` returns `ssn ***-**-6789 rest` (live-verified 10/12/2026). Bound both ends, as here, so the pattern cannot fire inside a longer number or an alphanumeric ID.

`splitByPattern` — splitting on a pattern

```
data record(t = "ERROR svc=api code=500")
| fieldsAdd parts = splitByPattern(t, "' '")
// live-verified: parts = ["ERROR", "svc=api", "code=500"]
```

All five are available inside the OpenPipeline DQL processor as well as at query time.

> **Sources:** Extraction and parsing commands (DT docs) — `parse` signature, the four parameters, and the field-override warning; String functions (DT docs) — `parse`, `parseAll`, `matchesPattern`, `replacePattern`, `splitByPattern`; OpenPipeline DQL functions (DT docs) — availability inside the DQL processor; DQL matcher in OpenPipeline (DT docs) — the matcher's *Functions* list is `matchesPhrase`, `matchesValue`, `isNotNull`, `isNull` and `duration` (read 10/02/2026); DPL Modifiers (DT docs) — *"negative look ahead - returns true if bytes forward from the current position do not match"*.

8. DPL Beyond Logs

DPL is almost always taught with logs, and both the documentation and this corpus's own examples reinforce that. **The restriction isn't real.**

`parse` operates on a *string expression*, not on a record type. Any string field on any record type can be parsed — span attributes, event fields, business-event payloads, entity names:

```
fetch spans, from: -10m
| filter contains(span.name, " ")
| limit 10
| parse span.name, "LD:verb ' ' LD:rest EOS"
| fields span.name, verb, rest
// live-verified 10/02/2026: "GET /api/currency?" → verb = "GET", rest =
"/api/currency?";
// "router frontend egress" → verb = "router", rest = "frontend egress"
```

Practical uses that have nothing to do with logs:

Record type	Use
Spans	Extract a tenant or route segment out of `url.path`; pull structure from a custom request attribute
Business events	Parse a composite order or SKU identifier into its parts
Events	Extract a change ticket number from a deployment event's title
Entity fields	Decompose a host or service naming convention into environment / region / role — see **FAQ-01**

That last one is a common request that teams usually solve with nested `substring()` calls. A naming convention is a grammar, and DPL parses grammars:

```
data record(host = "prd-use1-web-014")
| parse host, "LD:env '-' LD:region '-' LD:role '-' INT:instance"
// env = "prd", region = "use1", role = "web", instance = 14
```

> **Sources:** Extraction and parsing commands (DT docs) — defines `expression` as "a field or string expression to parse", with no record-type restriction stated. The Dynatrace documentation does **not**

> explicitly state that `parse` works on spans, events, or business events; the claim here rests on live-tenant execution against `fetch spans` (07/20/2026; re-run on `span.name` 10/02/2026) plus the string-expression signature. **Derived:** the applicability table is an authoring synthesis of that finding.

---

9. Where to Parse — Ingest vs. Query

The same pattern can run in two places, and the choice is architectural rather than stylistic.

	Ingest-time (OpenPipeline)	**Query-time (DQL `parse`)**
Runs	Once, as data arrives	Every time the query runs
Applies to	Records ingested **after** the rule is deployed	All records in retention, including historical
Changing it	Redeploy; no effect on data already stored	Edit the query; instantly retroactive
Wrong pattern costs you	Permanently unparsed data	Nothing — fix and re-run
Best for	Fields queried constantly; fields needed for bucket routing, security context, or metric extraction	Exploration, ad-hoc investigation, one-off analysis

The decision rule mirrors **FAQ-09**'s metric-vs-log-query logic: **how often will this field be read?** A field on a dashboard refreshing every minute should be parsed once at ingest. A field you need for a single incident should be parsed in the query and forgotten.

Two asymmetries matter more than the table suggests:

- **Ingest-time parsing is forward-only.** Deploy a pattern today and yesterday's records stay unparsed forever. Anything you might want historically must be parseable at query time regardless.
- **Ingest-time parsing is where the mistakes are expensive.** A query-time typo costs one re-run. A pipeline typo silently mis-populates every record until someone notices — and § 5 established that DPL failures don't announce themselves.

> **Forthcoming (SaaS 1.348, staged rollout planned from 09/22/2026):** a third place — DQL gains the ability to use DPL *"to parse file contents and shape the data at read time"* on files referenced through External Data Stores. This is pre-release information; verify it has reached your tenant. Until then, ingest and query time above are the two places DPL runs.

The practical sequence: develop at query time, promote to ingest time. Iterate in DPL Architect against real records until the pattern is right, confirm it at query scale, and only then move the settled pattern into a pipeline processor.

Limits that bound ingest-time patterns

Limit	Value
DQL processor script length	8,192 characters
Processor matching condition length	4,096 characters
Record size after processing	16 MB
Processors per pipeline	1,000

The Dynatrace documentation states **no** DPL-specific performance or DPS-consumption cost for pattern complexity. In community practice, teams treat pattern count and pipeline depth as the thing to keep an eye on rather than individual pattern complexity — verify against your own pipeline metrics before optimizing on assumption.

> **Sources:** OpenPipeline limits (DT docs) — the four limits in the table; What's new in SaaS 1.348 (DT docs) — *"use the Dynatrace Pattern Language (DPL) to parse file contents and shape the data at read time"* (pre-release, read 10/02/2026). **Derived:** the ingest-vs-query decision rule extends FAQ-09's read-frequency heuristic to parsing; the forward-only asymmetry follows from OpenPipeline processing records at ingest, which the docs describe but do not frame as a parsing-design constraint.

---

10. Diagnosing a Failed Pattern

Work down this list in order — it moves from loudest failure to quietest.

Step 1 — Is it a syntax error or a non-match?

They are completely different problems and DPL tells them apart clearly:

- **Syntax error** — the query fails outright: `The parsing pattern is invalid. Named pattern element 'EMAIL' is not valid`. A matcher name is wrong, or a required element is missing (`ARRAY requires a quantifier expression`). Check the name against § 4.
- **Non-match** — the query succeeds and every field is `null`. The syntax is fine; the pattern doesn't fit the text.

Step 2 — For a non-match, test the pattern against one known record

Shrink the problem. Copy one real line into `data record(...)` and rebuild the pattern from the left, one matcher at a time, checking the output after each addition. The first addition that turns a value into `null` is the culprit.

```
data record(t = "&lt;paste one real log line here&gt;")
| fieldsAdd step1 = parse(t, "IPADDR:ip"),
            step2 = parse(t, "IPADDR:ip ' - '"),
            step3 = parse(t, "IPADDR:ip ' - ' LD:user ' '")
```

Zero bytes scanned, instant feedback, no tenant data required.

Step 3 — Check the usual suspects

Symptom	Likely cause
Every record `null`	A literal that isn't in the text — different separator, tab vs. space, or the format varies more than your sample showed
Some records `null`	Genuinely optional fields — mark them `?`, or use alternation
One field holds a single character	Two `LD`s with nothing between them (§ 5, Rule 3), or an unquantified bracket group such as `[a-z]:x` (§ 4)

Symptom	Likely cause
Works on short lines, fails on long ones	The `{1,4096}` default cap on `LD` / `DATA`
Works on the first line of a multi-line record only	You want `DATA`, not `LD`
Timestamp is right but hours are off	Missing `timezone=` — DPL normalizes to UTC
Numeric field `null` for valid-looking numbers	`min=` / `max=` bounds rejecting them, or `INT` overflow — use `LONG`
Masking appears to do nothing	The pattern doesn't match, or the matcher doesn't exist — § 13

Step 4 — Measure the parse rate before you trust it

A pattern that works on your three samples may cover 60% of production. Quantify it:

```
fetch logs, from: -1h
| filter matchesPattern(content, "IPADDR ' - - [' DATA")
| summarize matched = count()
```

The trailing `DATA` matters: `matchesPattern` must match the whole string, so without it this count is 0 (§ 7). Compare against the unfiltered count. Anything short of the coverage you expected means the format varies — go find the variants before deploying to a pipeline.

> **Sources:** DPL Architect (DT docs) — coverage feedback as the intended validation loop. **Derived:** the four-step ladder and the symptom table are authoring syntheses of the live-verified behaviors in § 5 and § 4; the documentation does not provide a troubleshooting guide.

---

11. DPL Architect

DPL Architect is the pattern-development environment, and using it is the difference between DPL being pleasant and DPL being guesswork. Its purpose, per the documentation: extract fields from records, develop patterns faster, and **get instant**

feedback about the effectiveness and coverage of your patterns. That last point is what a hand-written pattern can't give you.

It is not a standalone app — it opens from inside the surfaces where you'd need it:

- **Notebooks** — add a *Logs → Fetch logs* section, run it, select a cell, choose **Extract fields**
- **Investigations** — several entry points from a record view

The loop:

1. Start from a query returning the records you actually want to parse
2. Write the pattern in the expression editor
3. Watch the **Match preview** highlight what the pattern consumes, live
4. Check the **Results** panel for the extracted fields as columns
5. **Add more records to the preview** — this is the step people skip, and it's the one that catches format variants
6. Insert the finished pattern back into the query

Two features worth knowing: patterns can be developed across **multiple tabs** for comparing approaches, and there are **preset patterns for 14+ technologies** — Apache, IIS, HAProxy, CoreDNS, Kubernetes, AWS, Google Cloud, and Microsoft formats among them. Check the presets before writing a pattern for any well-known format; the odds are good it already exists.

> **Sources:** DPL Architect (DT docs) — purpose statement, entry points, workflow, multi-tab support, and technology presets.

---

12. Recommended Approach

1. **Check for a dedicated matcher first.** `HTTPDATE`, `KVP`, `DQS`, `JSON`, `CREDITCARD`, and the DPL Architect technology presets replace a great deal of hand-written pattern. Reach for § 4 before writing anything.
2. **Translate regex deliberately, never mechanically.** Character groups port verbatim; `.*` → `LD` does not. Re-test every ported pattern against real records — a regex translation that "looks right" is exactly the shape of failure DPL doesn't report.

3. **Develop in DPL Architect against real records, never from memory.** Add records to the preview until you've seen the format's variants.
4. **Never place two `LD`s next to each other.** Always separate them with a literal or a non-optional matcher.
5. **State quantifiers explicitly whenever length matters** — both because the open-ended forms are capped (§ 6) and because the defaults differ by matcher: a bracket group takes one character, POSIX classes and `LD` up to 4,096.
6. **Prefix your export names** (`x_status`) so `parse` can't override existing semantic fields.
7. **Develop at query time, promote to ingest time.** Only move a pattern into a pipeline once it's settled and you've measured its parse rate.
8. **Measure coverage with `matchesPattern` before deploying**, and re-measure after — a pattern is a hypothesis about a format until you've counted.
9. **Test every masking pattern against sample data containing the thing you're masking.** Redaction that doesn't match fails silently and looks identical to redaction that works.

---

13. Common Gotchas

Inventing a matcher that doesn't exist. `EMAIL` , `URI` , `URL` , `HTTPMETHOD` , `HEXNUM` , `IPPREFIX` , and `DATETIME` all sound like they should be matchers. None of them are. This fails loudly at query time — but in an OpenPipeline processor a rule built on a non-existent matcher can be saved and simply never do its job. **If that rule was a masking rule, PII flows unredacted while the configuration says it's protected.** Verify every matcher name against § 4.

Two `LD`s with nothing between them. Covered in § 5 and repeated here because it produces plausible-looking wrong data rather than an error. `parse(t, "LD:x LD:y")` on `"alpha beta gamma"` returns `x = "a"` . *(live-verified)*

Assuming `+` is unbounded. It is capped — 4,096 per the docs, 8,192 observed (§ 6). Long records silently exceed it; set an explicit maximum.

Hand-rolling quoted strings as `'"' LD:x '"'` . Breaks on any escaped quote inside the value. Use `DQS` .

Hand-rolling the Apache timestamp. `HTTPDATE` produces byte-identical output to the nine-part `TIMESTAMP('dd/MMM/yyyy:HH:mm:ss Z')` spelling. *(live-verified)*

Forgetting `timezone=`. DPL normalizes to UTC. A local-time log without a timezone config produces timestamps offset by your UTC delta — data that looks fine until someone correlates it with something else.

Letting `parse` override a semantic field. Exporting to `status` or `duration` on a record that already has those fields overwrites them. DQL warns; the query still runs.

Testing masking with a pattern instead of with data. The only valid test is sample text containing a real example of what you're redacting, run through `replacePattern`, with the output inspected.

Deploying to a pipeline before measuring the parse rate. Ingest-time parsing is forward-only: records that arrive while the pattern is wrong are permanently unparsed.

Letting a character class match its own delimiter. `[a-z.]+ '.' [a-z]+` returns `null` on `john.doe` because the class eats the dot and DPL never backtracks to give it back. The most common cause of a pattern that matches nothing. *(live-verified)*

Writing `INT{3}` for a fixed-width number. Numeric matchers take only `*` and `+`. Use `[0-9]{3}`. This one at least errors rather than failing silently. *(live-verified)*

Porting a regex literally. `.*` → `LD` looks like a safe substitution and isn't: regex `.*` is greedy, `LD` is minimal. The ported pattern parses, returns values, and under-matches. See § 3.3.

Writing `.` and expecting a wildcard. In DPL `'.'` is a literal period. `\d`, `\w`, and `\s` don't exist at all — they're syntax errors, not silent no-ops. *(live-verified)*

Assuming `matchesPattern` takes a regex. Everything named `*Pattern` takes DPL. `like` takes SQL wildcards, `matchesValue` and `matchesPhrase` take a `*` glob with no mid-string support, and `contains` takes no wildcards at all. See § 3.6.

Assuming your masking language is portable. OneAgent-side masking is regex; OpenPipeline-side masking is DPL. A rule written for one will not fire in the other. See § 3.5.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace

documentation.